

CSA0315 – DATA STRUCTURES
CO5-AT1 - PSEUDOCODE WRITING TASK
(ALGORITHM DESIGN)

Faculty:

Dr. T. Rajesh Kumar

By:

Harshith G

192525394

1) Social Network Graph Analysis

Pseudocode:

BEGIN

1. Find Connected Components

visited $\leftarrow$ empty set

components $\leftarrow$ empty list

FOR each user u in V DO

 IF u is not in visited THEN

 component $\leftarrow$ empty list

 Create an empty queue Q

 Add u to visited

 Enqueue u into Q

 WHILE Q is not empty DO

 current $\leftarrow$ Dequeue(Q)

 Add current to component

 FOR each neighbor v of current DO

 IF v is not in visited THEN

 Add v to visited

 Enqueue v into Q

 END IF

 END FOR

```
        END WHILE
        Add component to components
    END IF
END FOR
```

2. Suggest Shortest Connection Path

```
distance[S]  $\leftarrow$  0
parent[S]  $\leftarrow$  NULL
visited  $\leftarrow$  empty set
Create an empty queue Q
Enqueue S into Q
Add S to visited

WHILE Q is not empty DO
    current  $\leftarrow$  Dequeue(Q)

    IF current = T THEN
        BREAK
    END IF

    FOR each neighbor v of current DO
        IF v is not in visited THEN
            Add v to visited
            distance[v]  $\leftarrow$  distance[current] + 1
```

```

        parent[v] ← current
        Enqueue v into Q
    END IF
END FOR
END WHILE

IF T is not in visited THEN
    PRINT "No connection path exists"
ELSE
    path ← empty list
    current ← T

    WHILE current ≠ NULL DO
        Insert current at beginning of path
        current ← parent[current]
    END WHILE

    PRINT "Shortest path:", path
    PRINT "Number of connections:", distance[T]
END IF

END

```

2) Task Scheduling Using a Directed Graph

Pseudocode:

BEGIN

1. Calculate In-Degree

FOR each task v in V DO

$\text{indegree}[v] \leftarrow 0$

END FOR

FOR each directed edge $u \rightarrow v$ in E DO

$\text{indegree}[v] \leftarrow \text{indegree}[v] + 1$

END FOR

2. Initialize Queue

Create an empty queue Q

FOR each task v in V DO

 IF $\text{indegree}[v] = 0$ THEN

 Enqueue v into Q

 END IF

END FOR

3. Generate Execution Order

order $\leftarrow$ empty list

WHILE Q is not empty DO

task $\leftarrow$ Dequeue(Q)

Add task to order

FOR each task next connected from task DO

indegree[next] $\leftarrow$ indegree[next] - 1

IF indegree[next] = 0 THEN

Enqueue next into Q

END IF

END FOR

END WHILE

4. Check for Cycles

IF size(order) = size(V) THEN

PRINT "No cyclic dependencies exist"

PRINT "Valid task execution order:", order

ELSE

PRINT "Cyclic dependency detected"

PRINT "No valid execution order exists"

END IF

END

3) Minimum Spanning Tree Using Kruskal's Algorithm

Pseudocode:

BEGIN

1. Initialize

MST $\leftarrow$ empty set

totalCost $\leftarrow$ 0

FOR each vertex v in V DO

 Make-Set(v)

END FOR

2. Sort Edges

Sort all edges in E in ascending order of weight

3. Select Minimum-Cost Connections

FOR each edge (u, v, weight) in sorted E DO

 IF Find(u) $\neq$ Find(v) THEN

 Add edge (u, v) to MST

 totalCost $\leftarrow$ totalCost + weight

 Union (u, v)

 ELSE

Skip edge

// Adding it would create a cycle

END IF

IF number of edges in MST = $|V| - 1$ THEN

BREAK

END IF

END FOR

4. Check Connectivity

IF number of edges in MST = $|V| - 1$ THEN

PRINT "All zones are connected"

PRINT "No redundant connections/cycles exist"

PRINT "Minimum installation cost:", totalCost

PRINT "Selected cable connections:", MST

ELSE

PRINT "Network cannot be fully connected"

PRINT "The given graph does not contain enough connections"

END IF

END